Simulation-Based Bayesian Estimation of Dynamic Models

Cameron Fen

May 24th, 2022

Outline

1 Introduction

2 The SNPE Algorithm

3 Results

How can we Remove the Barriers to Bayesian and Full Information Estimation in Macroeconomic Modeling?

- Maximum likelihood (MLE) and Bayesian estimation are better in many ways (efficiency, distributional inference etc.)
- There are two main barriers to Bayesian and MLE estimation:
  1. Bayesian/MLE estimation doesn't work with ⟨insert favorite solution method here⟩
  2. For the methods that are amenable, traditional Bayesian estimation techniques are slow to converge and inaccurate
- I will introduce an approach that will help address both issues for Bayesian and MLE estimation

Solution Method Compatibility

Solution Methods: A Brief Review

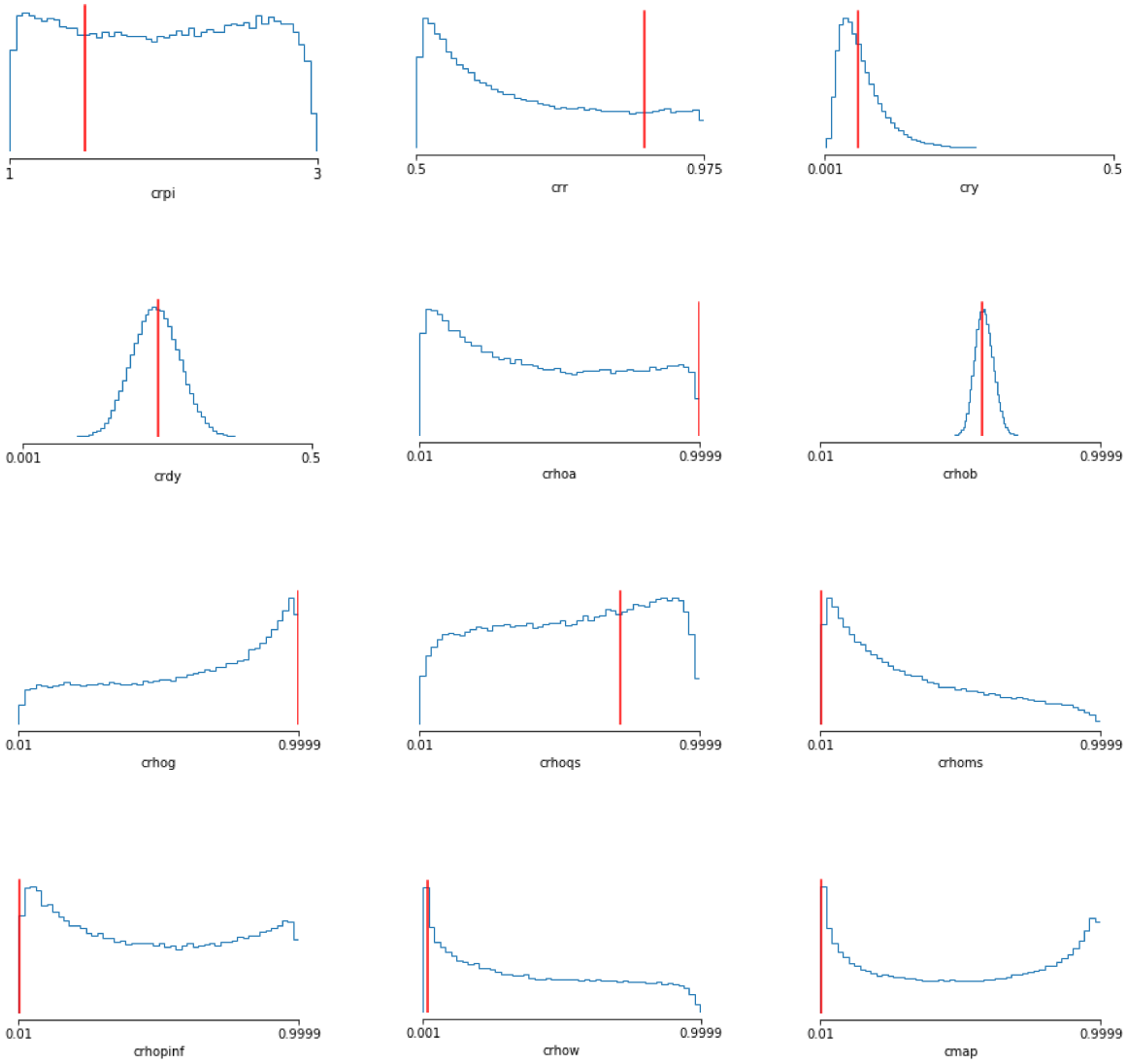
- Solution methods concern itself with solving for the correct policy function given parameters, estimation concerns itself with finding the set of parameters that when solved generate realistic data
- **Projection:** Uses a function approximator for control variables such that the policy function satisfies all the first order conditions of the structural model
- **Perturbation:** Uses a Taylor series approximation around the steady state to estimate a model.
  - Perturbation is the only method that yields a complementary likelihood function when estimated. As such it is the only solution approach conventional methods can Bayesian and MLE estimate
- **Value Function Iteration:** Generates estimates to the value function by using dynamic programming to iterate on the Bellman equation

Large Literature Attempting to Estimate Non-smooth Dynamic Models

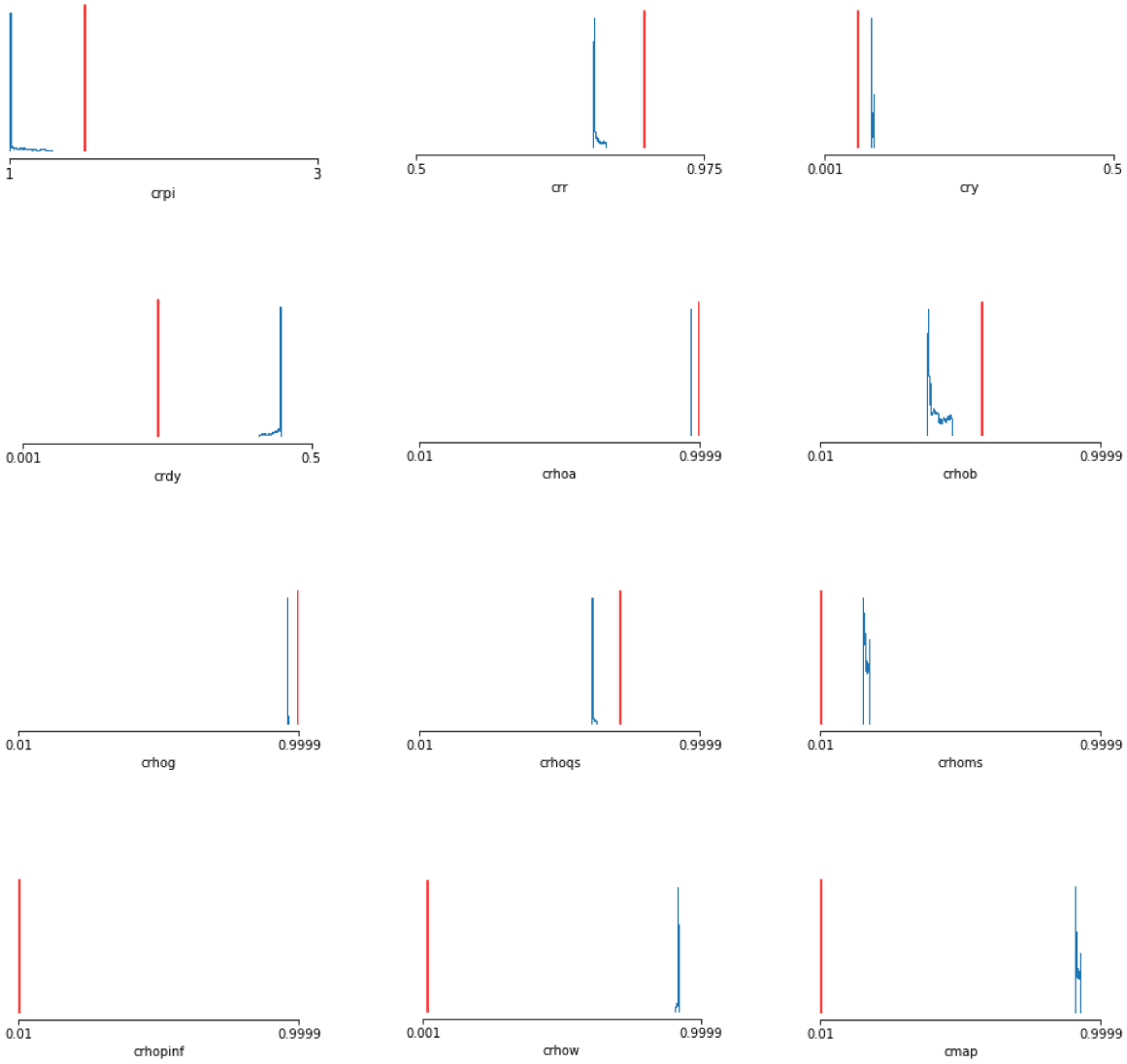
- Large literature solving non-smooth dynamic models
  - **s-S Models**: Arrow et. al. 1951, Caplin and Spulber, 1987, Bertola and Caballero 1990, House and Leahy 2004, Caplin and Leahy 2006, Kahn and Thomas 2008, etc
  - **Structural Finance**: He, Whited, and Guo 2021, Terry, Whited, and Zakolyukina, 2021, Taylor 2012, Whited and Guo 2006 etc.
- There is difficulty in building efficient estimation methods for these models, especially with a latent time component like macro models:
  - **Method of Simulated Moments**: McFadden 1989, Pakes and Pollard 1989, Duffie and Singleton 1990, etc.
  - **Efficient MSM, Indirect Inference, and Maximum Simulated Likelihood**: Smith Jr 1993, Gourierou, Monfort, and Renault 1993, Kristensen and Shin 2012, Lerman and Manski 1981, etc.
  - **Machine Learning**: Kaji, Manresa, and Pouliot 2020

Speed

What Good Posteriors Should Look Like:



What We Do Currently:



A Solution to Both Problems

Sequential Neural Posterior Estimation

- This paper advances an simulation-based approach from the deep learning literature, Sequential Neural Posterior Estimation (SNPE) that address both these barriers
  - Because this is a simulation based approach, can be used with any solution method even ones that don't generate likelihood functions
  - This approach also yields better posteriors faster than traditional Bayesian MCMC
- Outside of macro (IO, Applied Theory, Econometrics) this algorithm can estimate models with intractable likelihoods even with a latent state space time series structure

Outline

1 Introduction

2 The SNPE Algorithm

3 Results

Bayes Rule

- Bayes Rule:

$$P(\theta|X) = \frac{P(X|\theta)P(\theta)}{P(X)} \tag{1}$$

- θ are parameters, $P(\theta)$ prior, X data, $P(X|\theta)$ likelihood, $P(\theta|X)$ posterior

Sequential Neural Posterior Estimation: The Approach

- SNPE shares similarities with Kristensen and Shin (2012)

1. Sample $\theta \sim P(\theta)$
2. Sample $X' \sim P(X'|\theta)$ to generate (X', θ) pairs
3. Use a ~~kernel~~ normalizing flow density estimator for the conditional density $P(\theta|X')$
4. Then $P(\theta|X' = X)$ is the posterior

The Normalizing Flow Density Estimator

- Unlike the kernel density estimator Kristensen and Shin (2012) use:
  - A normalizing flow neural network density estimator can handle high dimensional data like data simulated from macro models, and...
  - It can estimate conditional densities
- Unlike K & S, this allows us to estimate macroeconomic models the right way in a MLE and Bayesian fashion
- Since this presentation is short, I will not discuss flows
- Hopefully this presentation can convince you that flows are powerful models and I'm happy to provide resources

Outline

1 Introduction

2 The SNPE Algorithm

3 Results

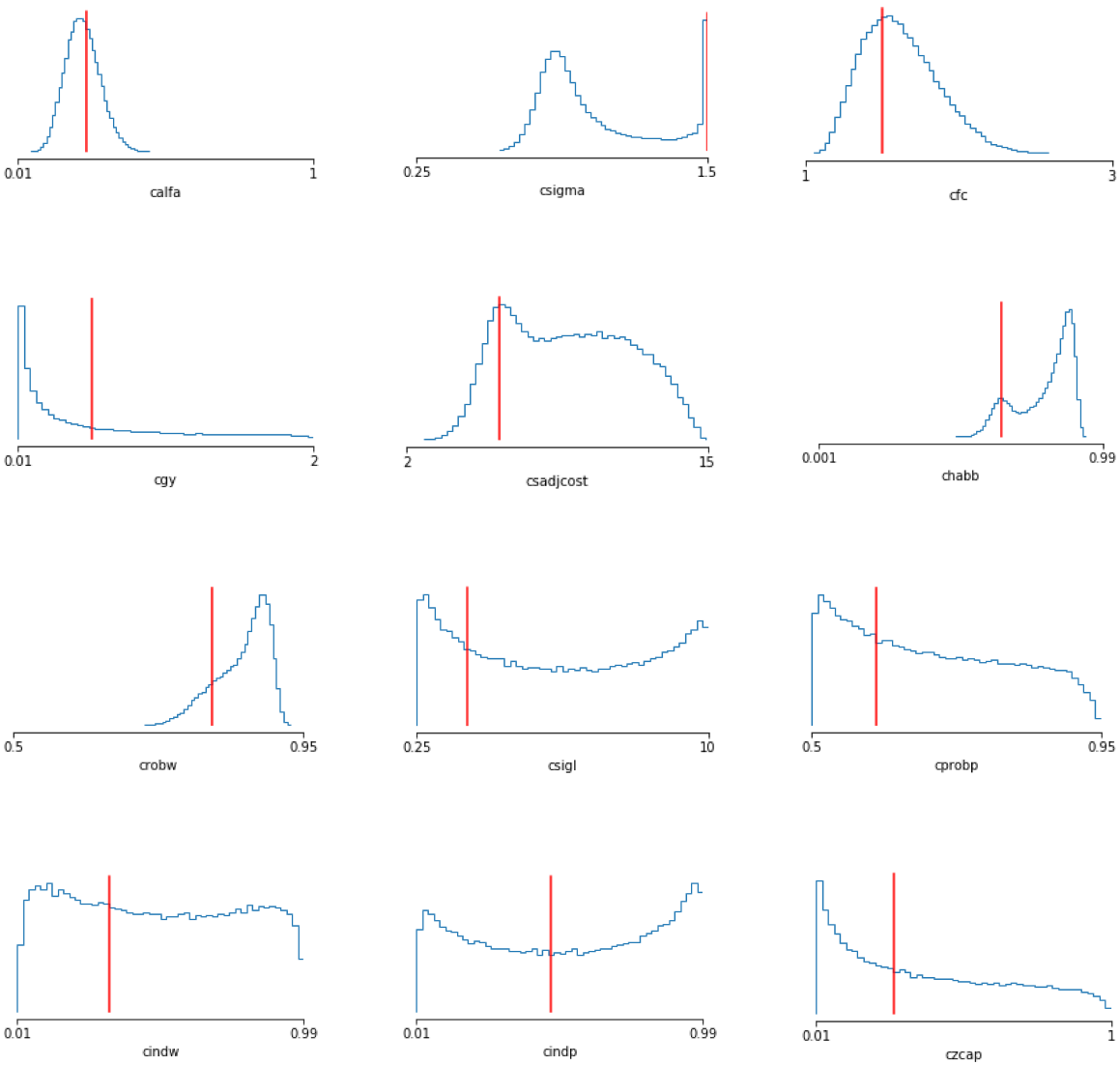
Models

Models	**SNPE**	**MCMC**	**K&S/GAN**	**Research Scale**
RBC Model:	✓	✓	∼	✗
Cash Flow Model:	✓	✗	∼	✗
Lucas Tree Model:	✓	✗	∼	✗
SW 2007 Model:	✓	✓	∼	✓
HANK Model:	✓	∼	∼	✓
Bequests Model:	✓	✗	∼	✓

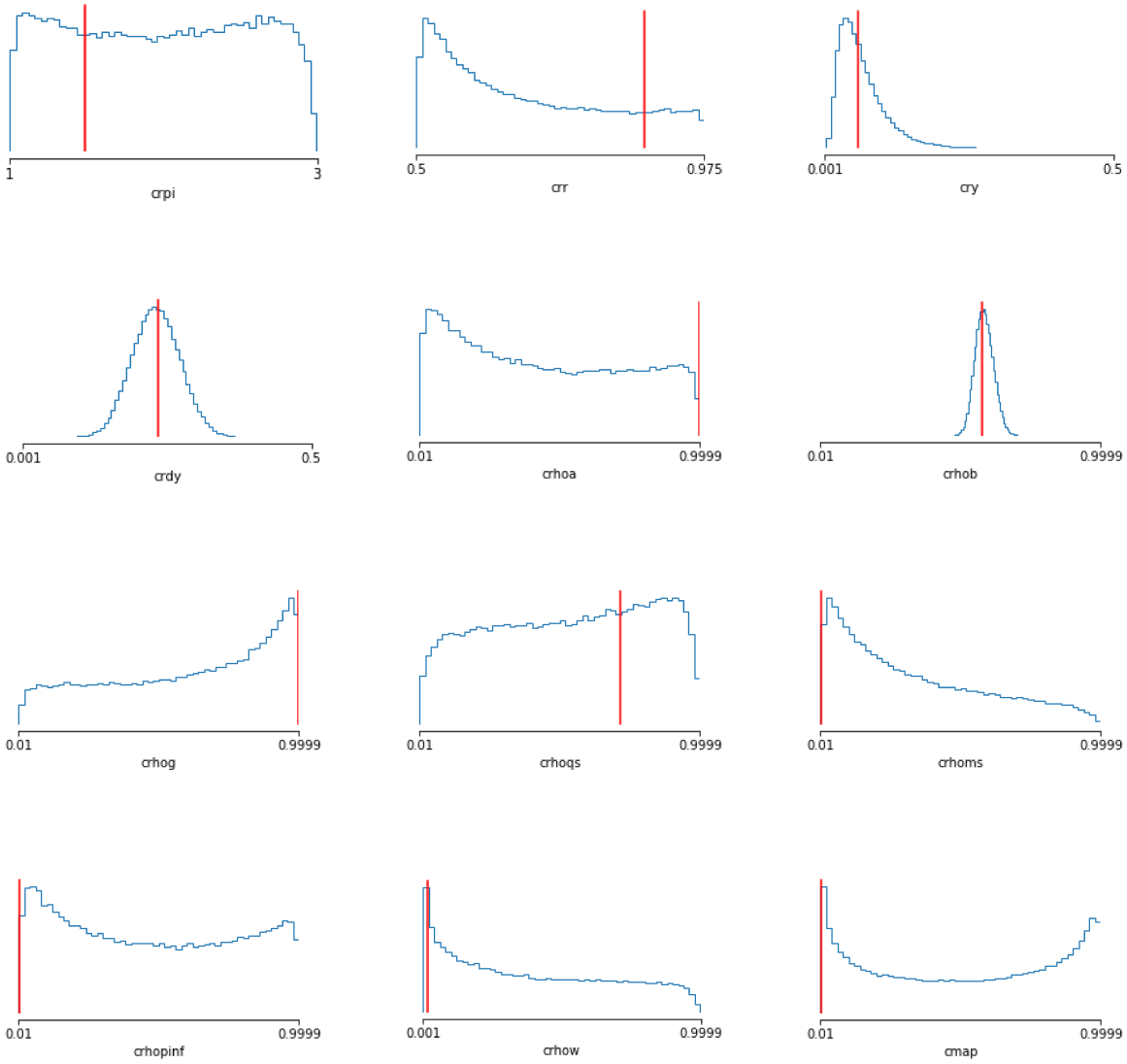
Case 1: Smets Wouters Model

- A New Keynesian model with Calvo price adjustment largely based on Christiano, Eichenbaum and Evans (2005)
- 36 parameters all estimated in a Bayesian fashion
- 7 observed variables and 7 autoregressive shocks
- Contains a flexible model economy to calculate potential GDP along with a sticky price economy

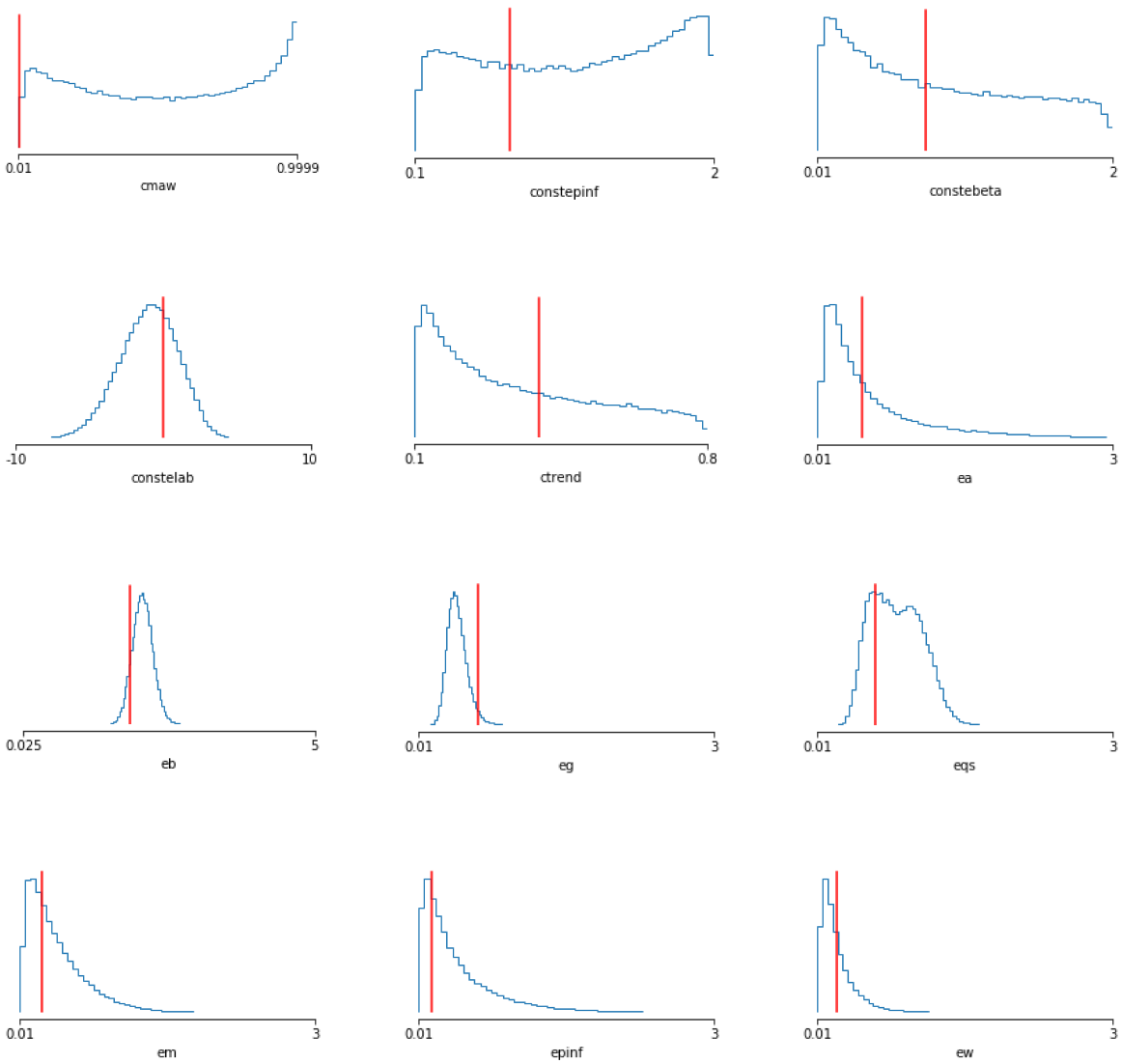
Smets-Wouters SNPE after 500K iterations: Params 1-12



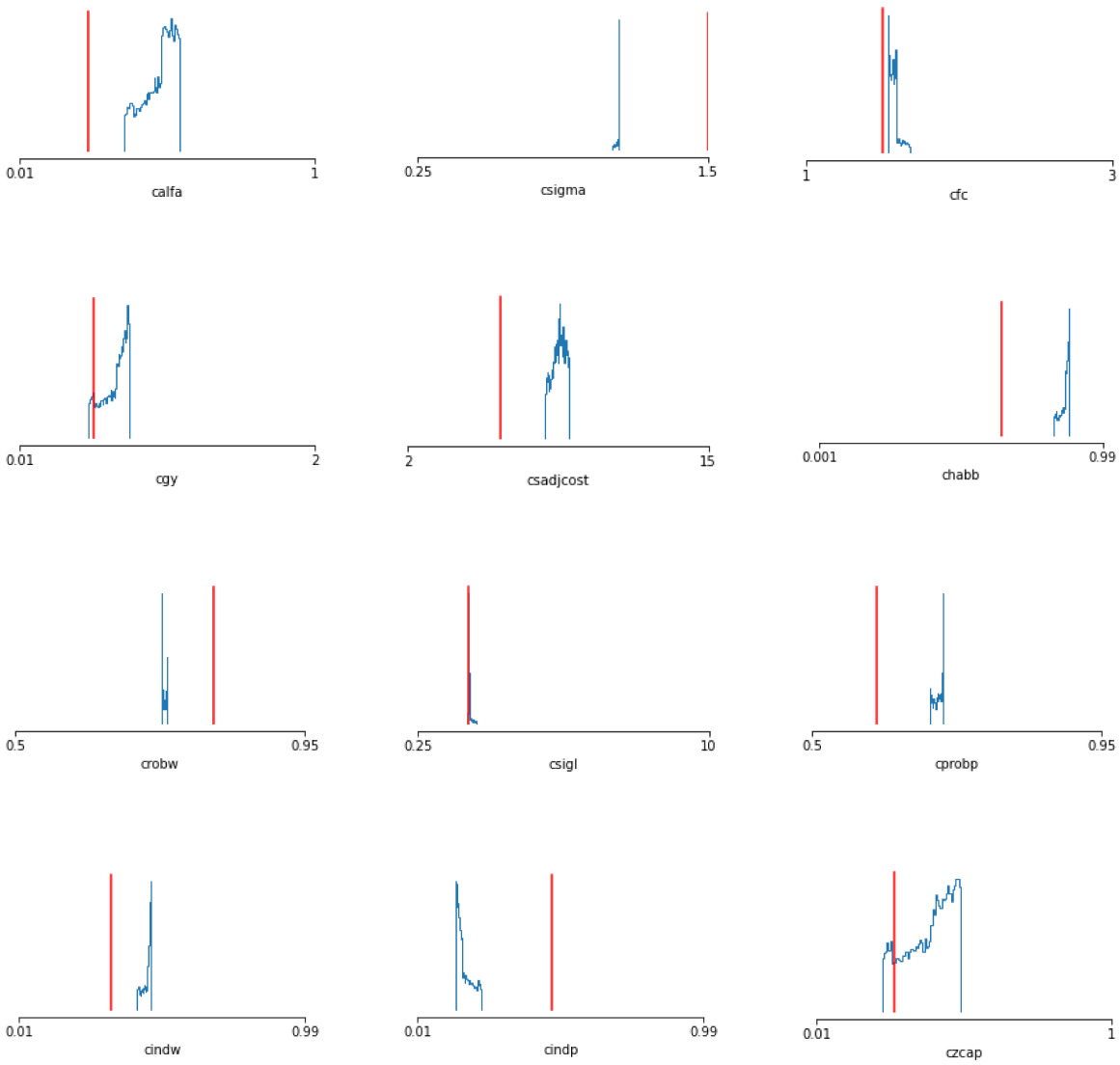
Smets-Wouters SNPE after 500K iterations: Params 12-24



Smets-Wouters SNPE after 500K iterations: Params 24-36



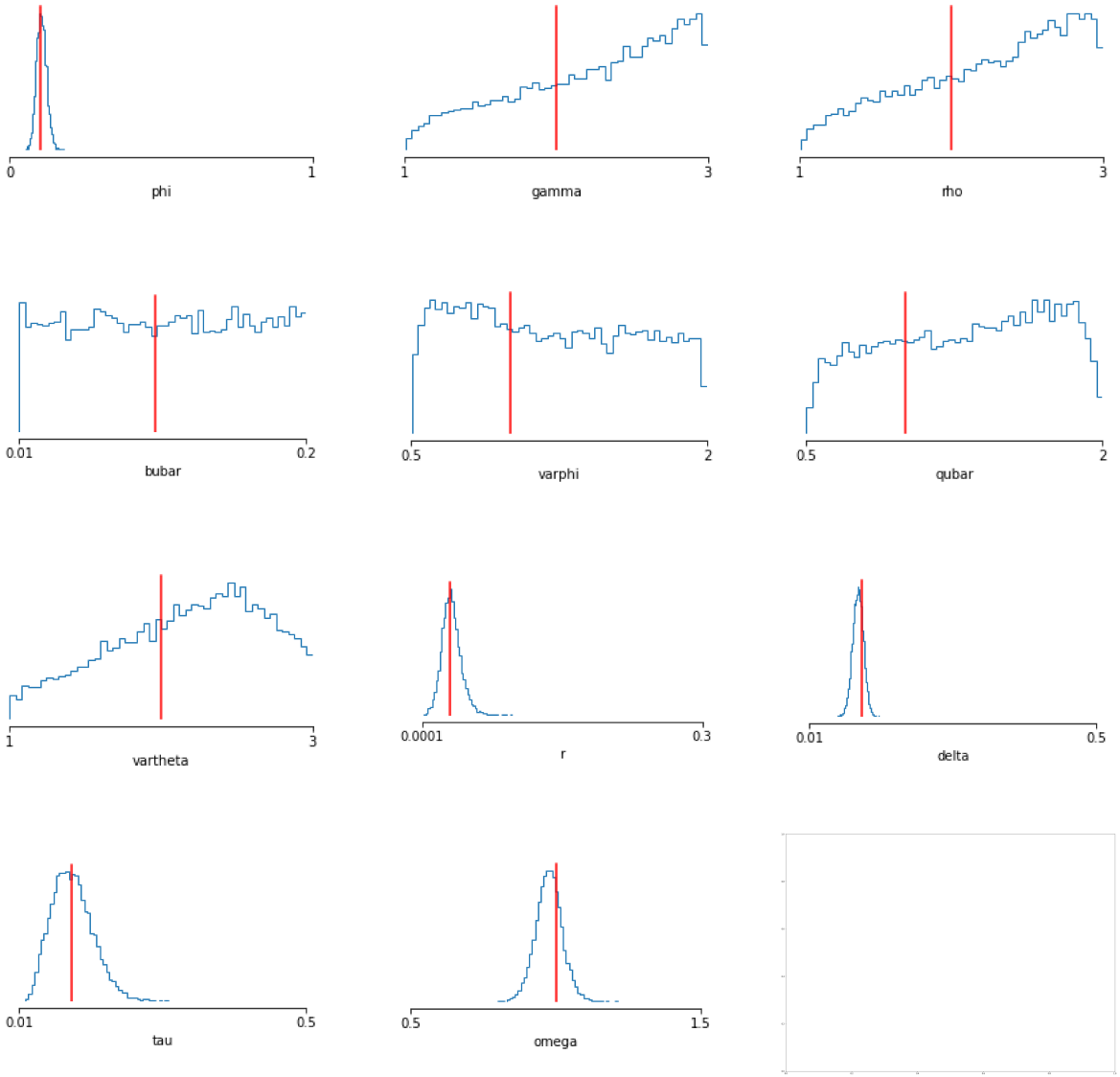
Smets-Wouters MCMC after 10M iterations: Params 1-12



Case 2: Bequests Value Function Iteration Model

- This is a bequests model where there are two assets, liquid and illiquid assets
- Liquid assets can be used in consumption, illiquid asset decays over time but provides a flow benefit
- Kink in the policy function where one switches from liquid to illiquid assets
- Solved via value function iteration

11 Parameter Value Function Iteration Model



Conclusion

- Extending the previous literature, I provided a method to estimate macroecnomic models in a Bayesian and MLE fashion
  - Since this method is a simulation based approach, works with all solution methods, not just perturbation like MCMC
  - Even for perturbation based models, yields posteriors that are better and faster than MCMC
- I advocate adopting the normalizing flow neural network density estimator for more applications in macroeconomics
- I showed empirical results where the estimation procedure converges to the true parameters or reproducible posteriors with better results than MCMC

Thank You

camfen@umich.edu

Appendix

The Normalizing Flow (Rezende and Mohamad 2015)

- The normalizing flow is a underused, if not novel, technique in economics
- A normalizing flow chains together stacks of bijectors so that the output of one bijector becomes the input of the next bijector
- The particular bijector I use is the neural autoregressive bijector (Huang et. al. 2018)
- As the name implies a bijector is a bijective function that maps (typically distributions) from $R^n \rightarrow R^n$
- Additionally the bijector is invertable not only in theory but in practice
- The nuts and bolts, the next couple slides, are difficult to explain in a short seminar for more information see: https://blog.evjang.com/2018/01/nf1.html

Bijectors

- Given an N dimensional vector of inputs ranging from $i = 1...n$, the outputs of a neural autoregressive bijector is defined as:

$$y_1 = \sigma(a * z_1 + b) \tag{2}$$

$$y_i = \sigma(\alpha(z_1...z_{i-1})z_i + \beta(z_1...z_{i-1})) \tag{3}$$

- Here $\alpha(.)$ and $\beta(.)$ are feed forward neural networks that only depend on $z_{1:i-1}$
- You can easily see that you can calculate the forward direction: y from z and the inverse direction: z from y

Bijectors: Discussing Sigma

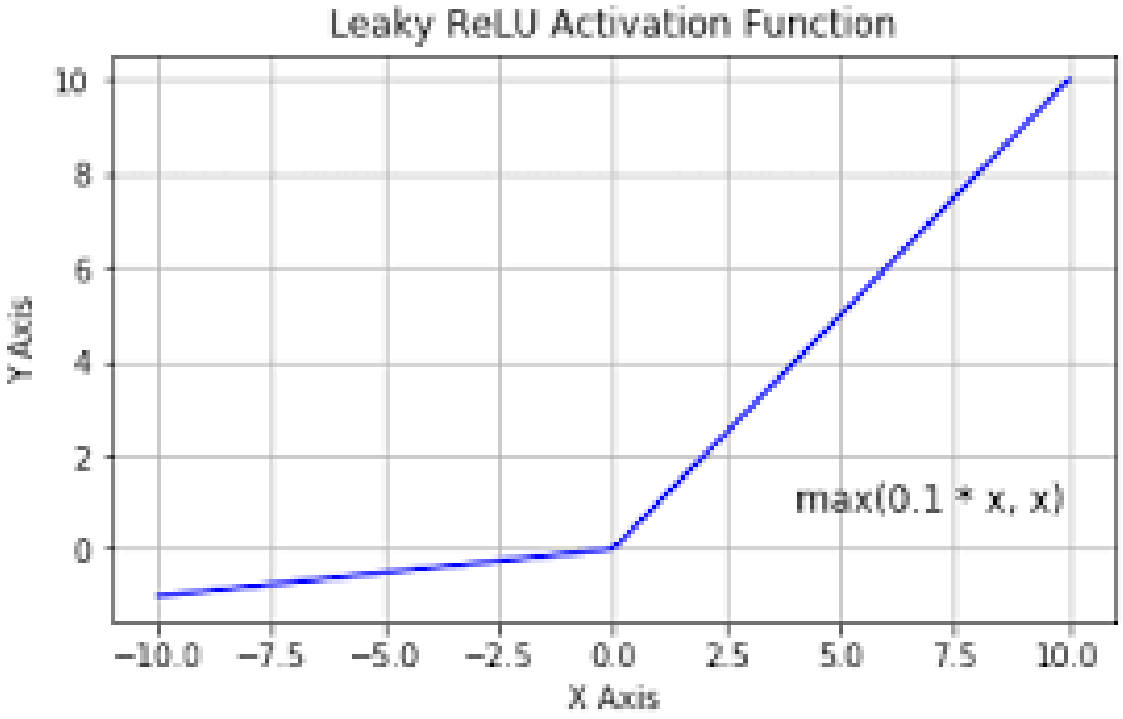
- Given an N dimensional vector of inputs ranging from $i = 1...n$, the outputs of a neural autoregressive bijector is defined as:

$$y_1 = \boldsymbol{\sigma}(a * z_1 + b) \tag{4}$$

$$y_i = \boldsymbol{\sigma}(\alpha(z_1...z_{i-1})z_i + \beta(z_1...z_{i-1})) \tag{5}$$

- Here $\alpha(.)$ and $\beta(.)$ are feed forward neural networks that only depend on $z_{1:i-1}$
- You can easily see that you can calculate the forward direction: y from z and the inverse direction: z from y

Sigma: The Leaky ReLU



Bijectors: Inverting an Affine Function

- Given an N dimensional vector of inputs ranging from $i = 1...n$, the outputs of a neural autoregressive bijector is defined as:

$$y_1 = \sigma(\boldsymbol{a} * z_1 + \boldsymbol{b}) \tag{6}$$

$$y_i = \sigma(\alpha(z_1...z_{i-1})z_i + \beta(z_1...z_{i-1})) \tag{7}$$

- Here $\alpha(.)$ and $\beta(.)$ are feed forward neural networks that only depend on $z_{1:i-1}$
- You can see that you can calculate the forward direction: y from z and the inverse direction: z from y

Bijectors: Inverting an Affine Function

- Given an N dimensional vector of inputs ranging from $i = 1...n$, the outputs of a neural autoregressive bijector is defined as:

$$\frac{\sigma^{-1}(y_1) - \boldsymbol{b}}{\boldsymbol{a}} = z_1 \tag{8}$$

$$y_i = \sigma(\alpha(z_1...z_{i-1})z_i + \beta(z_1...z_{i-1})) \tag{9}$$

- Here $\alpha(.)$ and $\beta(.)$ are feed forward neural networks that only depend on $z_{1:i-1}$
- You can see that you can calculate the forward direction: y from z and the inverse direction: z from y

Bijectors: Inverting a Normalizing Flow

- Given an N dimensional vector of inputs ranging from $i = 1...n$, the outputs of a neural autoregressive bijector is defined as:

$$y_1 = \sigma(\boldsymbol{a} * z_1 + \boldsymbol{b}) \tag{10}$$

$$y_i = \sigma(\boldsymbol{\alpha}(\boldsymbol{z_1...z_{i-1}})z_i + \boldsymbol{\beta}(\boldsymbol{z_1...z_{i-1}})) \tag{11}$$

- Here $\alpha(.)$ and $\beta(.)$ are feed forward neural networks that only depend on $z_{1:i-1}$
- You can see that you can calculate the forward direction: y from z and the inverse direction: z from y

Bijectors: Inverting a Normalizing Flow

- Given an N dimensional vector of inputs ranging from $i = 1...n$, the outputs of a neural autoregressive bijector is defined as:

$$y_1 = \sigma(\boldsymbol{a} * z_1 + \boldsymbol{b}) \tag{12}$$

$$y_i = \sigma(\boldsymbol{\alpha}_{\boldsymbol{z}_{1:i-1}} z_i + \boldsymbol{\beta}_{\boldsymbol{z}_{1:i-1}}) \tag{13}$$

- Here $\alpha(.)$ and $\beta(.)$ are feed forward neural networks that only depend on $z_{1:i-1}$
- You can see that you can calculate the forward direction: y from z and the inverse direction: z from y

Bijectors: Inverting a Normalizing Flow

- Given an N dimensional vector of inputs ranging from $i = 1...n$, the outputs of a neural autoregressive bijector is defined as:

$$\frac{\sigma^{-1}(y_1) - \boldsymbol{b}}{\boldsymbol{a}} = z_1 \tag{14}$$

$$\frac{\sigma^{-1}(y_i) - \boldsymbol{\beta}_{\boldsymbol{z}_{1:i-1}}}{\boldsymbol{\alpha}_{\boldsymbol{z}_{1:i-1}}} = z_i \tag{15}$$

- Here $\alpha(.)$ and $\beta(.)$ are feed forward neural networks that only depend on $z_{1:i-1}$
- You can see that you can calculate the forward direction: y from z and the inverse direction: z from y

The Change of Variable

- The flow learns a mapping from a known "base" distribution like a multivariate normal random variable to a "target" distribution one wants to estimate
- One trains the flow by modifying the parameters the model to learn the correct change of variable from the base distribution to the target distribution
- If the base distribution has a pdf of $P(z)$, the output distribution of a flow f has a pdf of $P(y) = P(z)|det(df/dz)|^{-1}$, based on a change of variable formula
  - The Jacobian term follows the same logic as the canonical 1st year econometrics example of a change of variable from a normal to a chi-squared distribution
- Thus you can perform density estimation by maximizing the likelihood of the data samples occurring under the flow

Example: Estimating the Likelihood of a Point

- Let's say we want to estimate the likelihood of a point $y = (1.5, -1.5)$ under a flow
- Procedure:
  1. Use the ability of bijectors to perform inversion to find the z point that would generate $y = (1.5, -1.5)$
  2. Say this point is $z = (0.2, 1.0)$. Then calculate $P(z = (0.2, 1.0))$ since one knows the base distribution by construction
  3. Evaluate the inverse of the determinant of the Jacobian at $z = (0.2, 1.0)$
  4. Derive the likelihood by putting the parts together to get $P(y) = P(z)|det(df/dz)|^{-1}$
  5. Knowing this point, maximize the likelihood, $P(y)$, by taking gradient steps in the direction of steepest increase.

Importance Sampling

- In practice one often samples and re-estimates the posterior multiple times
- In the first round one samples $\theta \sim P(\theta)$
- But in later rounds one wants the proposal distribution for θ to be as close to the posterior as possible
- Thus in later rounds of estimation one usually samples $\theta \sim P'(\theta|X = X)$, the estimate of the posterior obtained in the preceding round
- In order to correct for sampling from the "wrong" distribution, instead of estimating just the normalizing flow density estimator $f_\phi(\theta|x)$, one estimates the density estimator along with an importance weighting factor, $f_\phi(\theta|x)\dfrac{p^{(i-1)}(\theta)}{p(\theta)}$ on samples (X, θ).

The Entire Algorithm: SNPE

Algorithm 1: SNPE Algorithm

Input: Simulator $p(x|\theta)$, prior $p(\theta)$, data x_0, flow $f_\phi(x|\theta)$, Rounds R, Samples S;

Initialize: Posterior $p^{(0)(\theta)} = p(\theta)$, data set D $= \{\}$;

for $i \leftarrow 1$ **to** R **do**

  Sample $\theta^{(n)} \sim p^{(i-1)}$ for $n = 1...S$ with Monte Carlo;

  Simulate $x^{(n)} \sim P(x|\theta^{(n)})$ for $n = 1...S$;

  Concatenate data $D = D \cup \{x^{(n)}, \theta^{(n)}\}_{n=1}^{S}$;

  while $f_\phi(x, \theta)$ *not converged* **do**

    Sample $\{x^{(i)}, \theta^{(i)}\}_i^B \sim D$ from D;

    Train $f_\phi(\theta|x)\dfrac{p^{(i-1)}(\theta)}{p(\theta)}$ on $\{x^{(i)}, \theta^{(i)}\}_i^B$;

  end

  Update posterior $p^{(i)(\theta)} = f_\phi(\theta|x_0)$;

end

Model of Firm Capital Formation

- From Toni Whited and Emil Lakkis
- Same as the RBC model, except profits after investment is the utility function

$$U(K_t, I_t, Z_t) = \sum_t \beta^t * (\pi(K_t, Z_t) - I_t) \tag{16}$$

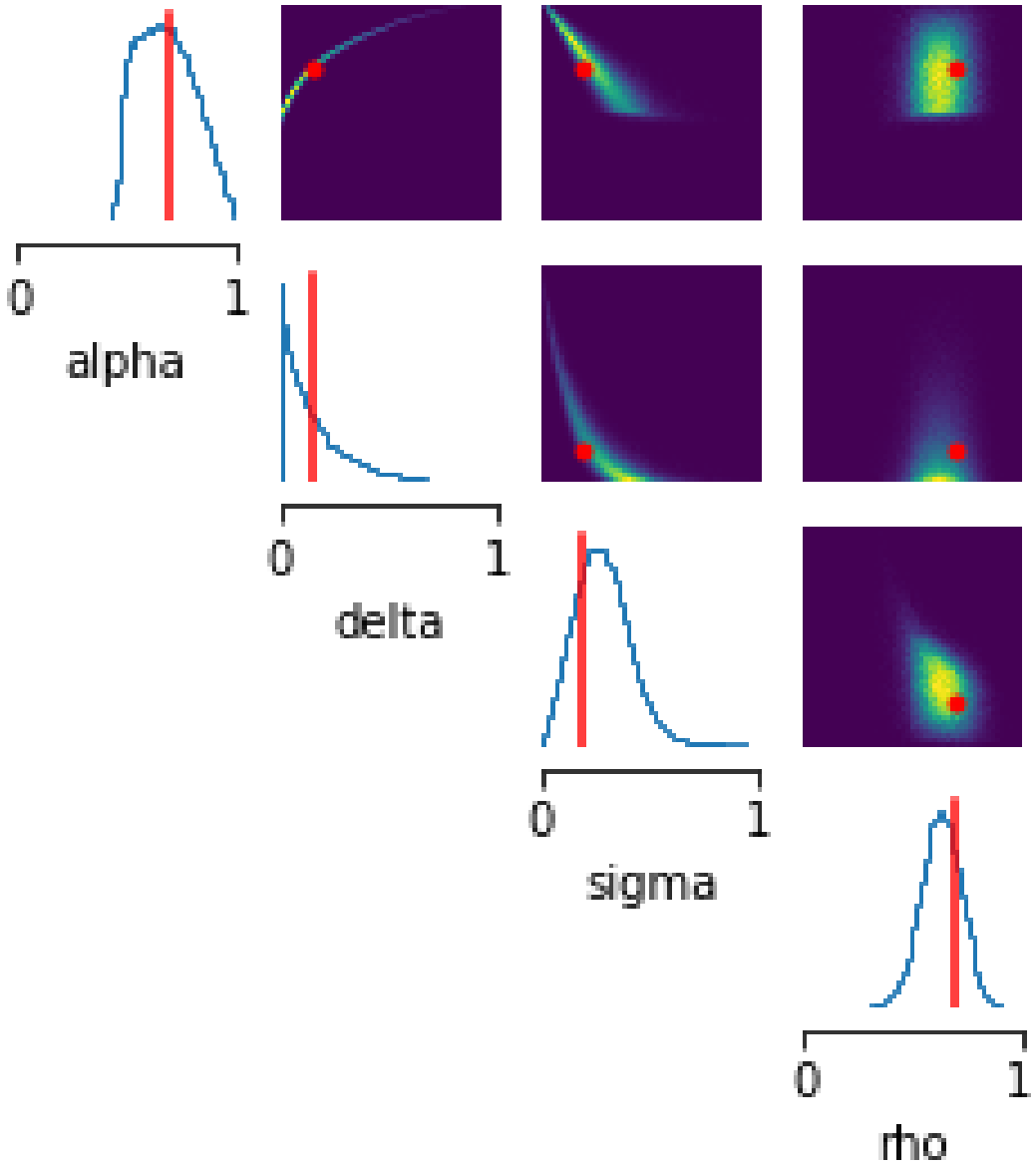
$$K_t = (1 - \delta)K_{t-1} + I_t \tag{17}$$

$$logZ_t = \rho logZ_{t-1} + \epsilon_t \tag{18}$$

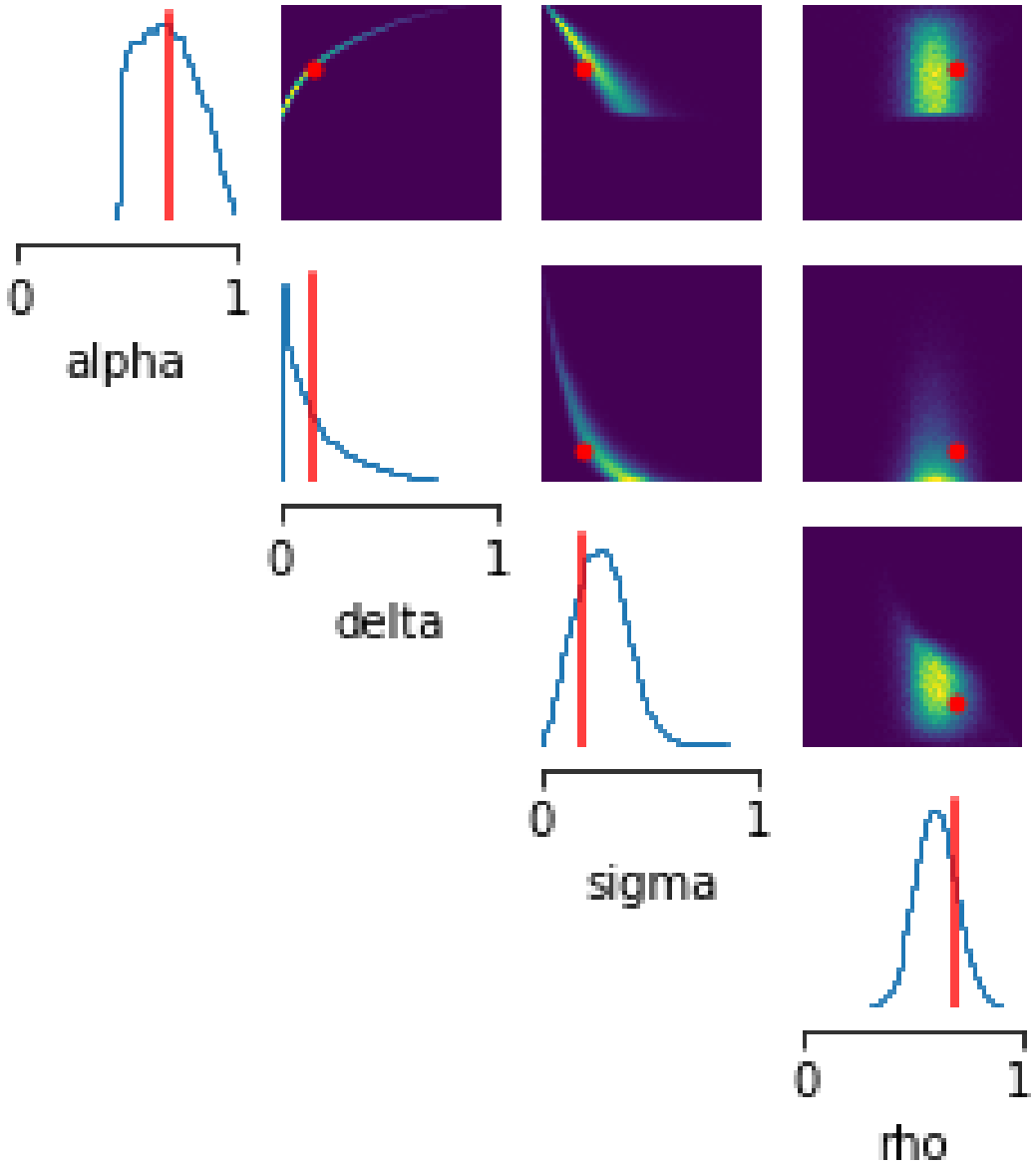
$$\pi(K_t, Z_t) = Z_t K_t^{\alpha} \tag{19}$$

$$\tag{20}$$

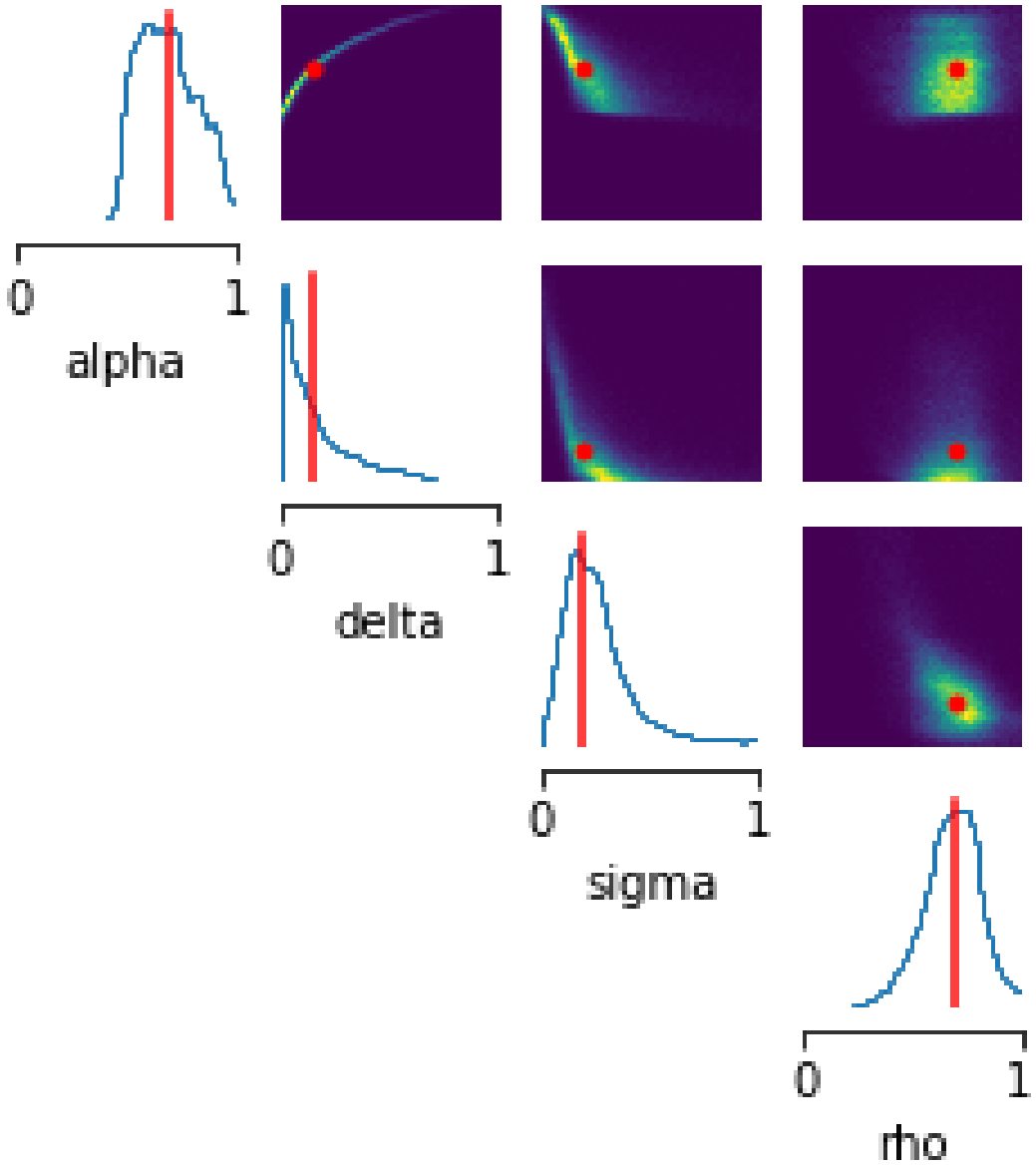
Value Function Iteration with Dense Net Embedding



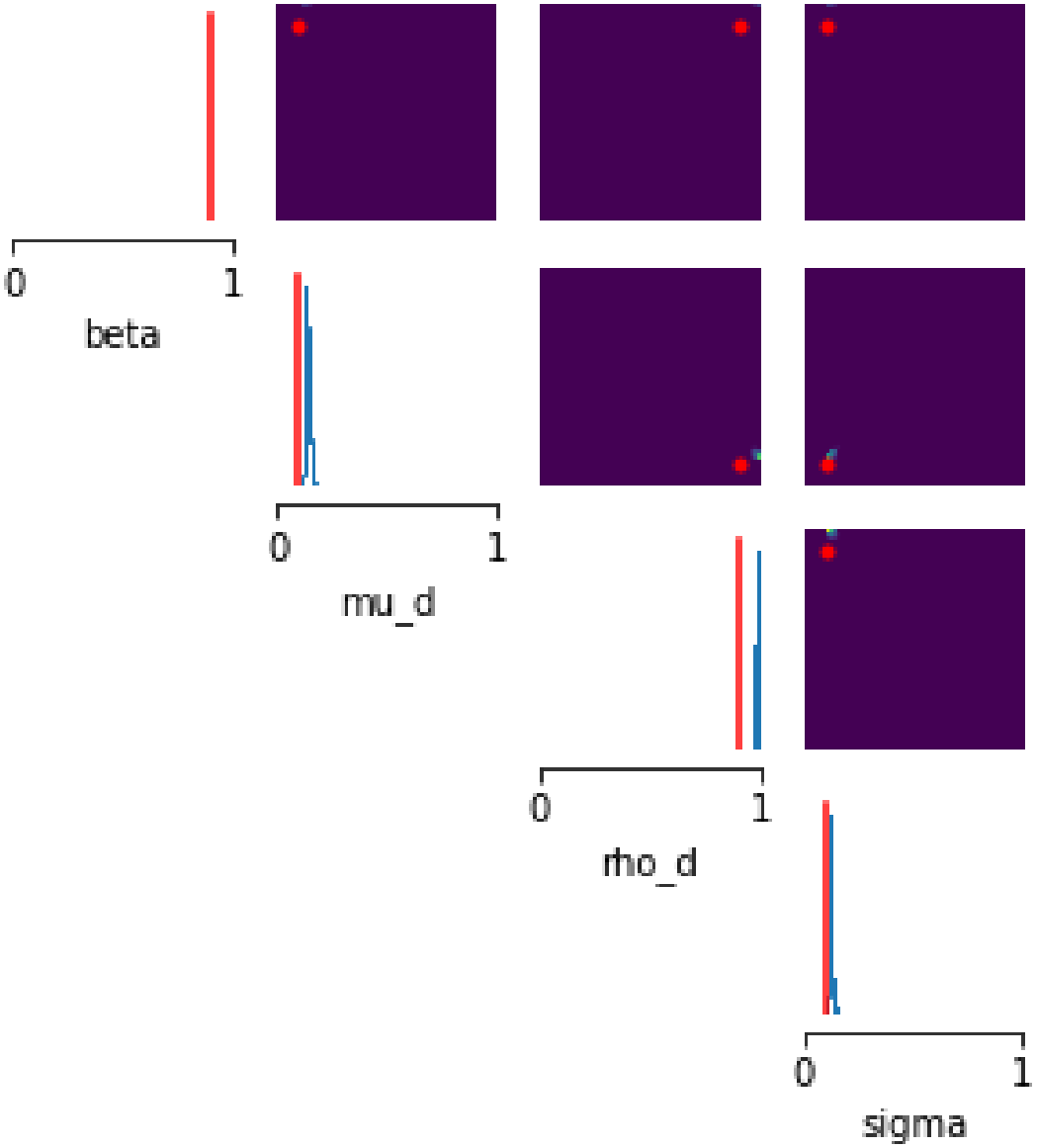
Value Function Iteration with RNN Embedding



Value Function Iteration with GAN Density Estimator



Lucas Tree Projection Model



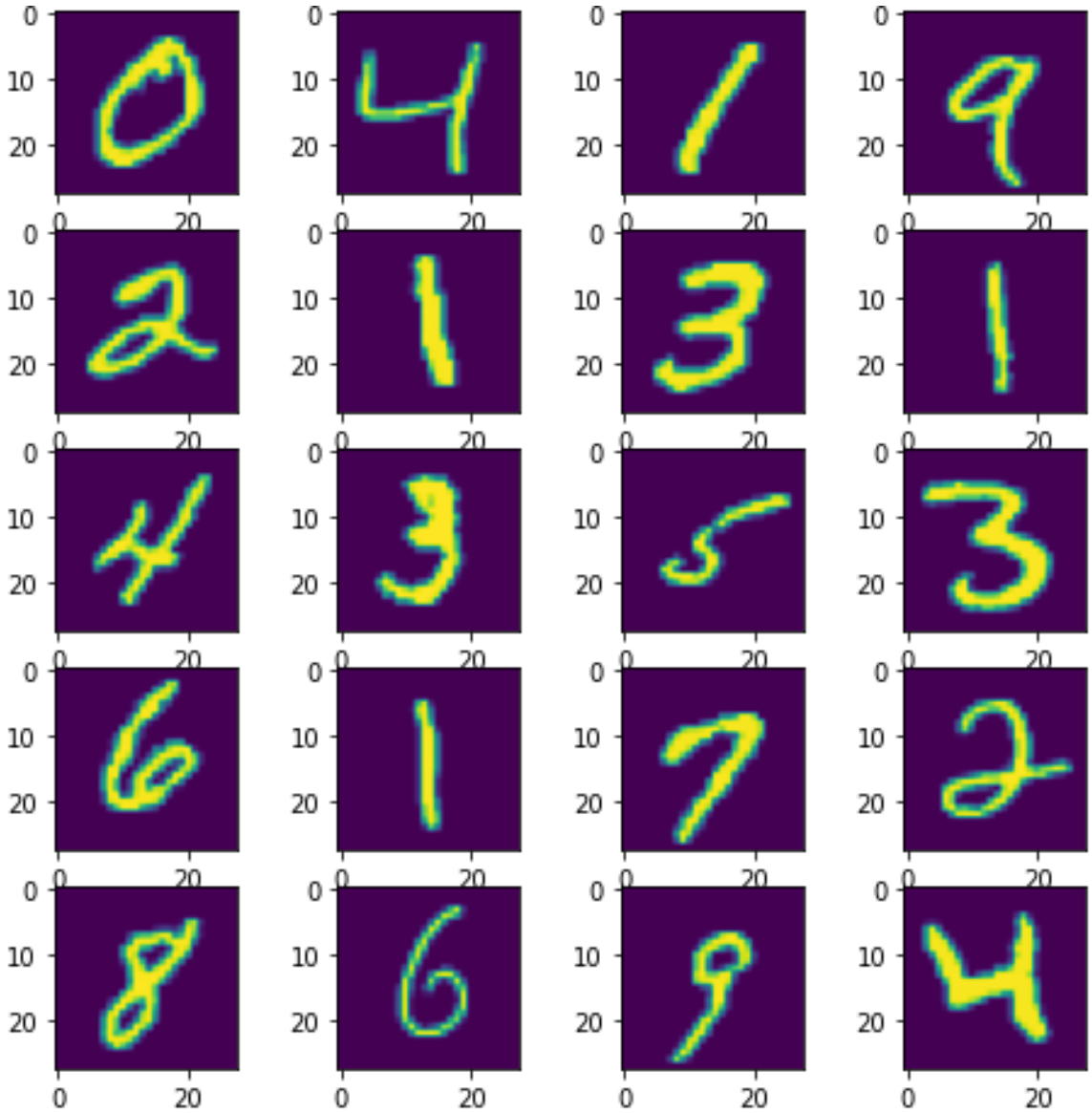
Outline

1 Introduction

2 The SNPE Algorithm

3 Results

Normalizing Flow Example



Change of Variable Proof

Theorem (Radon Nikodym Theorem for Normalizing Flows)

1. Normalizing Flow is a Valid Change of Variable (Sketch)

- The neural autoregressive flow as set up is a bijection, if one sets the parameters of the $\alpha(.)$ and $\beta(.)$ neural networks correctly.
- As long as all the parameters are bounded, the neural autoregressive flow is Lipschitz, which implies that the input and output are absolutely continuous with respect to one another.
- Thus, one can apply the Radon-Nikodym theorem on the change in measure in both directions on the base and target distribution.
- This means the change of variable formula in the forward direction is: $P(y) = P(z)|det(df/dz)|^{-1}$, as discussed above.

Flow Universal Approximator Proof

Theorem (Flow Universal Approximator)

1. The Normalizing Flow is a Universal Approximator of Monotonic Functions (Sketch)

- First I show a step function can universally approximate any monotonic function using an argument similar to universal approximators of simple functions in measure theory.
- Then I show that a logistic regression can arbitrarily approximate any step function.
- Since $\alpha(.)$ and $\beta(.)$ are feed forward neural networks which are universal approximators of continuous functions, a neural autoregressive flow can arbitrarily approximate any logistic regression.
- Thus flows universally approximate monotonic functions

Flow Universal Approximator Proof

Theorem (Monotonic Functions and Distribution Mapping)

2. Monotonic Functions Can Universally Map Any Continuous Distribution to Any Other (Sketch)

- This proof implies that any distribution can be transformed to any other distribution via a monotonic function that is a change in variables. The proof resembles Sklar's theorem for copulas.
- First one can transform any distribution to a uniform distribution via a monotonic function by recognizing that the CDF of any distribution is exactly that transform.
- Likewise the inverse CDF allows one to transform to a uniform distribution from any other distribution via a monotonic change in variable
- Since one can map any distribution to a uniform and a uniform to any distribution, all distributions can be mapped to one another via a monotonic transform.

RBC Model Description

- From Alisdair McKay: https://alisdairmckay.com/Notes/HetAgents/index.html
- Standard CRRA utility:

$$u(C_t) = E \sum_t \beta^t \frac{C_t^{1-\gamma}}{1-\gamma} \tag{21}$$

- Cobb-Douglas production, standard law of motion, and productivity AR in logs

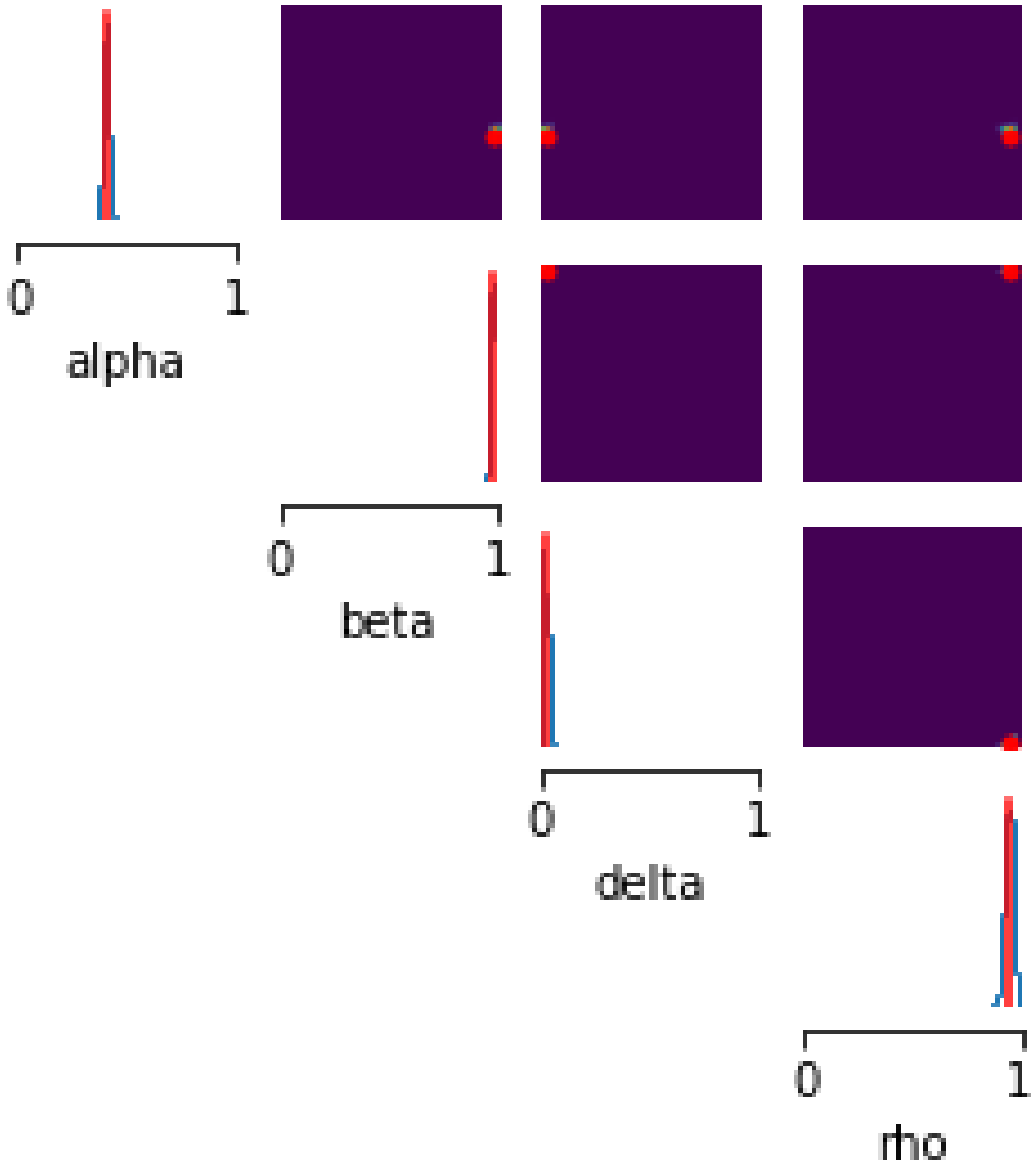
$$Y_t = Z_t K_t^{\alpha} \bar{L}^{1-\alpha} \tag{22}$$

$$K_t = (1-\delta)K_{t-1} + Y_t - C_t \tag{23}$$

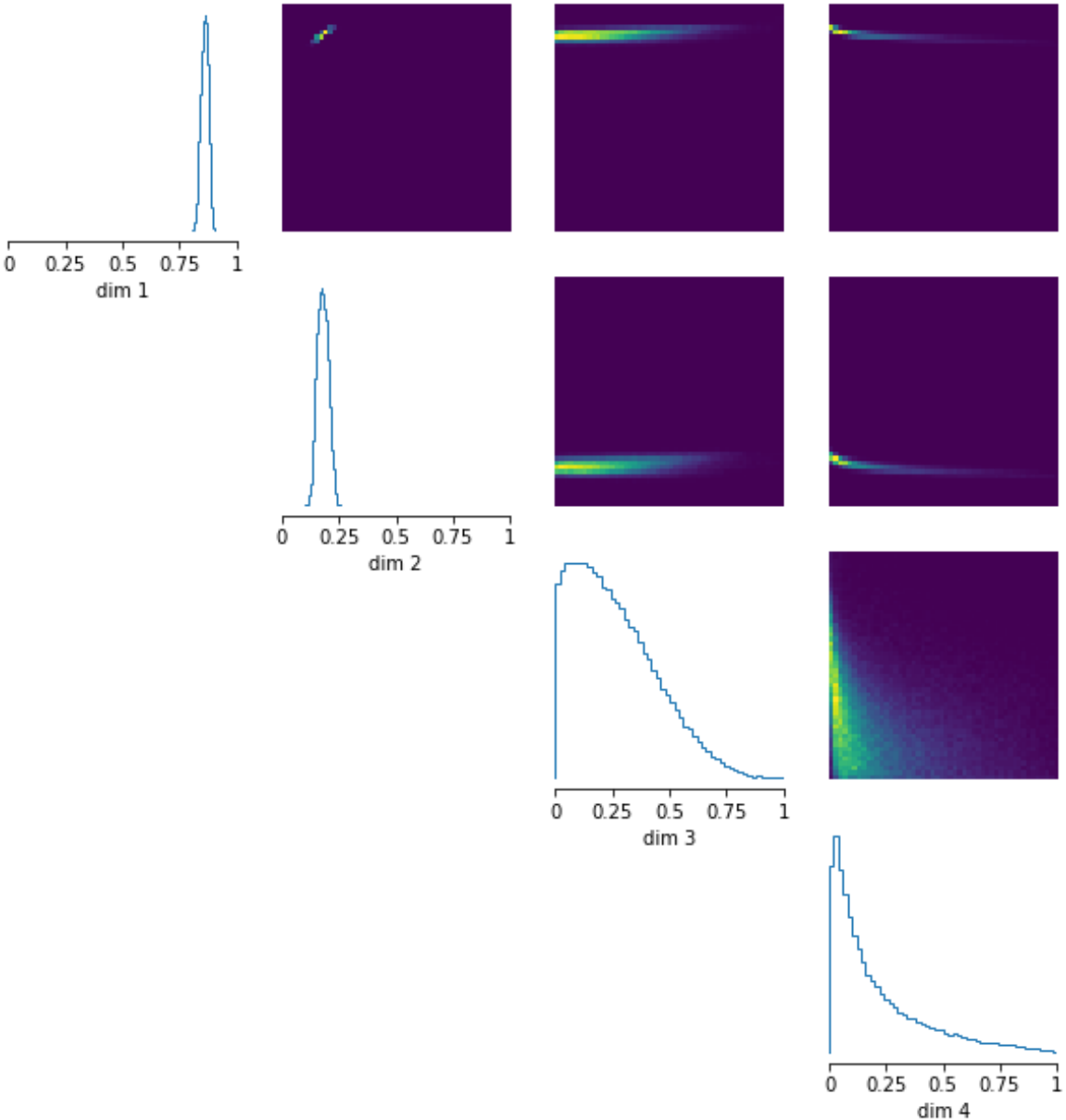
$$logZ_t = \rho logZ_{t-1} + \epsilon_t \tag{24}$$

$$\tag{25}$$

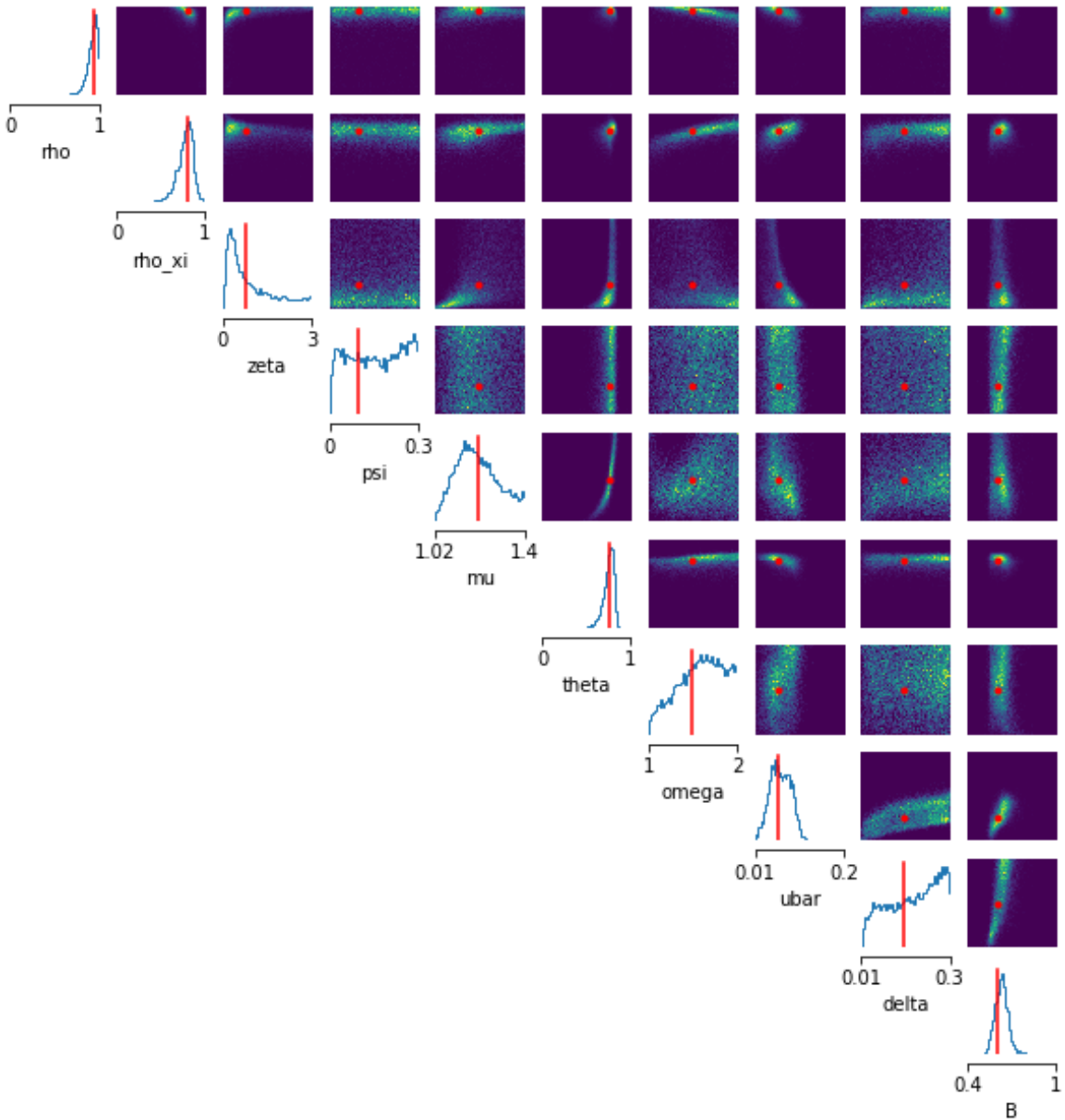
Simulated Data RBC



Real Data RBC



HANK Model

- From Alisdair McKay
- Wealth is the main distributional state
- Production function linear in productivity and labor
- Standard New Keynesian backbone (Dixit-Stiglitz aggregator, Calvo pricing etc.)
- Labor with search, government with bonds

10 Parameter HANK Model